

¿Aportan información las invariantes geométricas de los grafos de programa por encima del análisis semántico clásico?

VALEN: un estudio falsable sobre OWASP Benchmark 1.2

Proyecto VALEN

Borrador de trabajo v0.4 — no revisado por pares

Abstract

VALEN es un *verificador neuro-simbólico de invariantes estructurales*: un agente autónomo que extrae especificaciones de seguridad (invariantes de autorización, ciclos de máquinas de estado, precondiciones de taint) desde el código con un LLM y las descarga con métodos formales (Z3, homología persistente/dirigida, un núcleo mecanizado en Lean 4) — emitiendo solo *witnesses verificables por máquina bajo la semántica modelada*. Este artículo plantea una única pregunta integral y la responde con una frontera precisa: *¿dónde aportan las invariantes estructurales de los grafos de programa información de seguridad, y dónde no?*

El lado negativo de la frontera es preciso. En las fallas de inyección lineal, donde un servlet plano y casi acíclico no tiene curvatura ni topología persistente que explotar y el taint clásico ya es casi óptimo, los priors estructurales no aportan: sobre OWASP Benchmark 1.2 (2740 casos) el taint crudo ordena mejor el sink vulnerable (MRR 0.894, AUC 0.895, supera al campo fusionado con $p = 5 \times 10^{-5}$, estimador add-one sobre 20000 permutaciones); las señales espectral (0.496) y topológica (0.500) están en el azar y la curvatura por debajo (0.376). Las hipótesis H1/H2/H3/H5 quedan *no soportadas*. El lado positivo es donde viven los defectos de mayor impacto del software moderno — BOLA/IDOR (CWE-639), autorización ausente (CWE-862) y ciclos de estado destructivos (reentrancia, deadlocks) — que son *estructurales*: el dato infractor es legítimo, así que el análisis de taint clásico es ciego ante ellos, y solo la *forma* de fronteras de privilegio y transiciones de estado los delata. Allí la misma maquinaria es la única señal: en un corpus curado de 39 casos de BOLA/IDOR, el detector estructural de *autorización ausente* alcanza recall 1.0 / precisión 0.72 / F1 0.837 mientras el taint puntúa 0 (sus 7 falsos positivos son el límite de sobre-aproximación documentado; un witness de ownership en Z3 separa la autorización a nivel de objeto de la mera autenticación; en datos reales — OWASP crAPI, 9 endpoints BOLA/BFLA documentados — una señal de referencia a objeto a nivel de API alcanza recall 1.0 / precisión 0.90 donde el chequeo de auth-ausente puntúa 0, porque todo endpoint BOLA *está* autenticado); la homología de caminos dirigida GLMY, evaluada sobre 12 grafos de estado, alcanza recall 1.0 / precisión 0.75 frente al 0.5/0.5 de la homología simetrizada que supera. El resultado es una única cuenta — el mismo grafo tipado, álgebra de privilegios, homología dirigida y verificador Z3 — de *qué* relaciones de programa hacen informativas a las invariantes estructurales. Contribuimos esta cuenta, un arnés de oráculo de priorización, un pipeline Java reproducible con witnesses Z3, validación externa en Juliet y CVEs reales, y una mecanización en Lean 4 de la solidez de la abstracción de taint y de la parte de no-escalada de la invariante de autorización.

1 Motivación

Los pipelines de análisis estático basados en reglas y los semánticos son frágiles: pasan por alto vulnerabilidades novedosas y ahogan al analista en falsos positivos, por problemas de composición, priorización, caminos complejos y gaps de modelado. Faltan dos capacidades:

- (1) Una *representación unificada y cuantitativa* de la “estructura del programa” sobre la cual proyectar y fusionar muchas señales *independientes*.
- (2) Un *ciclo de razonamiento* que convierta “región sospechosa” en “hallazgo verificado bajo la semántica codificada” — el LLM aporta la intuición, la matemática aporta la verificación.

VALEN proporciona ambas.

2 La idea central

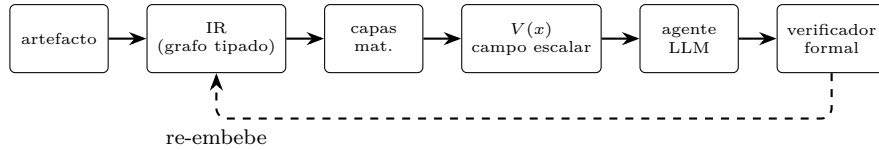


Figure 1: El pipeline: un artefacto se convierte en un grafo tipado, enriquecido con estructura matemática en un campo escalar de vulnerabilidad $V(x)$, que un agente LLM navega; un verificador formal confirma o refuta cada hipótesis y el resultado se re-embebe en el valen.

Modelamos a los candidatos a vulnerabilidad como *violaciones de invariantes estructurales*, no como patrones puramente sintácticos (una invariante de autorización rota, un ciclo persistente, un cuello de botella de curvatura negativa, un flujo de taint que cruza un corte espectral de confianza). Cada capa detecta un *tipo* distinto de violación de invariante, y el campo fusionado $V(x)$ ordena las regiones para que el agente las inspeccione.

3 La Representación Intermedia (IR)

Definición 1 (Grafo de programa). Un *grafo de programa* es una tupla $G = (V, E, \tau_V, \tau_E, \omega)$ donde

- V es un conjunto finito de *nodos* (módulos, funciones, bloques, sentencias, llamadas, variables, fuentes, sumideros);
- $E \subseteq V \times V$ es un conjunto de *aristas* dirigidas;
- $\tau_V : V \rightarrow \mathbb{N}_K$ asigna a cada nodo un *tipo* de $K = \{\text{módulo, función, bloque, sentencia, llamada, asignación, variable, parámetro, fuente, sumidero, gate}\}$;
- $\tau_E : E \rightarrow \mathcal{T}$ asigna a cada arista un *tipo* $\mathcal{T} = \{\text{control, dato, llamada, taint, confianza, auth}\}$;
- $\omega : E \rightarrow \mathbb{R}_{\geq 0}$ asigna a cada arista un *peso* no negativo (por defecto 1).

La IR es la única fuente de verdad. Cada adaptador (código fuente, binario, API web, agente LLM) debe producir un grafo de programa; cada análisis lo consume. El esquema es agnóstico al lenguaje y serializable a JSON (las capas Python y Rust comparten el mismo esquema).

Nombre. VALEN = *Verificación y Lógica Activa, Neuro-simbólica*.

4 Capas matemáticas

Para cada capa fijamos un subgrafo por tipo de arista τ_E , y luego computamos estructura.

4.1 Capa espectral (teoría algebraica de grafos)

Para un tipo de arista t , definimos la *adyacencia* simétrica A_t y el *grado* D_t , y el *Laplaciano combinatorio*

$$L_t = D_t - A_t.$$

L_t es semidefinido positivo; su espectro $0 = \lambda_1 \leq \lambda_2 \leq \dots$ codifica la estructura global:

- λ_2 es la *conectividad algebraica* (valor de Fiedler). Su autovector f_2 — el *vector de Fiedler* — induce un corte canónico $S^+ = \{v : f_2(v) \geq 0\}$.
- El *embedding espectral* $\Phi^{(d)}(v) = (f_2(v), f_3(v), \dots, f_{d+1}(v))$ mapea nodos a $\mathbb{R}^d$.
- La desigualdad de Cheeger acota el cociente de cuello de botella $h(G)$ por $\frac{\lambda_2}{2} \leq h(G) \leq \sqrt{2 d_{\max} \lambda_2}$.

Señal 1 (anomalía espectral). El error de reconstrucción de un nodo bajo una reconstrucción de eigenmap de bajo rango, y su distancia al corte de Fiedler, se usan como puntajes de anomalía.

4.2 Capa topológica (homología persistente)

Del grafo de programa construimos una *filtración* de complejos simpliciales (p. ej. un complejo de Vietoris–Rips o de bandera sobre la métrica de caminos más cortos, o una filtración guiada por ω). La homología persistente produce, para cada dimensión k , un *diagrama de persistencia* Dgm_k de pares nacimiento–muerte. β_0 cuenta componentes, β_1 cuenta ciclos independientes; los puntos lejos de la diagonal tienen alta persistencia. El algoritmo **Mapper** (Singh–Mémoli–Carlsson) construye un *nervio* navegable a partir de una función filtro $f : V \rightarrow \mathbb{R}$, produciendo el mapa que da nombre al proyecto.

4.3 Capa geométrica (curvatura discreta)

La *curvatura de Ollivier–Ricci* de una arista (u, v) es

$$\kappa(u, v) = 1 - \frac{W_1(m_u, m_v)}{d(u, v)},$$

con medidas de paseo aleatorio perezoso m_u, m_v y distancia de Wasserstein-1 W_1 . La curvatura negativa marca *embudos* por los que pasan muchos caminos más cortos. La *curvatura de Forman–Ricci* (sin pesos) es $\kappa_F(u, v) = 4 - \deg(u) - \deg(v)$, marcando de forma barata los nodos de alto grado.

4.4 Capa algebraica (retículo de taint y orden de autorización)

Retículo de taint. El dominio abstracto es el retículo de conjuntos de etiquetas de taint $\mathbb{T} = (2^{\mathcal{K}}, \subseteq, \cup, \cap)$, con $\perp = \emptyset$ (limpio) y $\top$ (manchado). Forma una conexión de Galois (α, γ) con la semántica recolectora concreta; la solidez se prueba en §4.6. *Orden de autorización.* El modelo de autorización es deliberadamente minimal — no se necesita teoría de categorías. Sea $F : V \rightarrow \mathbb{N}$ una *función de nivel de privilegio* sobre puntos de programa (distintos puntos pueden compartir nivel; una arista

no tiene por qué subirlo); una arista $e = (u, v)$ es *arista auth* si y solo si $F(u) < F(v)$ (escalada estricta), y E_A es el conjunto de aristas auth. Un *sanitizador* en un punto mapea toda etiqueta a limpio. **Invariante de autorización:** un flujo de taint que alcanza un sink es una *violación* cuando su camino cruza una arista auth y ningún sanitizador del segmento de cruce restaura la invariante. Esto se enuncia con precisión en §4.6 (la parte de no-escalada está mecanizada).

4.5 Capa formal (ejecución simbólica y model checking)

La ejecución simbólica calcula una condición de camino Φ y un estado simbólico σ ; un solucionador SMT comprueba $\text{SAT}(\phi_{\text{malo}})$; un modelo es un witness bajo la semántica codificada, no una prueba de explotabilidad end-to-end. La alcanzabilidad es la fórmula CTL **EF** ϕ_{malo} ; la propagación de taint es un autómata finito cuyo lenguaje aceptado es exactamente el conjunto de caminos fuente $\rightarrow$ sumidero.

4.6 Núcleo formal: invariante de autorización y solidez

Semántica small-step de la IR. Un estado de ejecución mapea variables a valores concretos; la relación de transición $v \xrightarrow{\sigma} v'$ avanza por la IR. La semántica recolectora concreta de taint $\mathcal{C}$ mapea cada punto al conjunto de ejecuciones que llevan un valor no confiable allí. El análisis abstracto de §4 es la función A calculada por las funciones de transferencia; el par (α, γ) satisface $\alpha(\mathcal{C}(v)) \sqsubseteq A(v)$ y $\mathcal{C}(v) \subseteq \gamma(A(v))$.

Proposición 1 (solidez de la abstracción de taint). *Si el análisis abstracto reporta una variable limpia, ninguna ejecución concreta la mancha.*

Esquema de demostración. Por inducción en la secuencia de transiciones, con la invariante “taint concreto $\sqsubseteq$ taint abstracto” en cada estado. La base vale para las variables de entrada; el paso vale porque cada función de transferencia es monótona y sobre-aproxima su contraparte concreta (las asignaciones unen las etiquetas de los operandos, los sanitizadores mapean a limpio, los joins acotan superiormente el merge de ramas). La garantía es condicional a que el *summary* del sanitizador corresponda a su semántica real: un summary incorrecto (que afirme limpiar lo que no limpia) rompe la solidez. Este es exactamente el enunciado mecanizado **prop1_soundness** (§4.6), probado para una abstracción booleana concreta; extenderlo al retículo completo de etiquetas es directo pero se asume aquí.

Proposición 2 (violación de la invariante de autorización). *Sea $F : V \rightarrow \mathbb{N}$ la función de nivel de privilegio, y sea $p = e_1 \circ \dots \circ e_n : s \rightarrow k$ un camino. Denotemos $\text{Auth}(p)$ las aristas auth de p y $\text{San}(p)$ los sanitizadores en p . Entonces: (i) si $\text{Auth}(p) = \emptyset$ entonces $F(k) \leq F(s)$ (sin escalada de privilegio); (ii) si una arista auth $e_i = (v_i, v_{i+1})$ yace en p y ningún sanitizador neutraliza la etiqueta antes de ese cruce, entonces la ejecución contiene un cruce de frontera de privilegio controlado por el atacante.*

Demostración. (i) Cada arista no-auth cumple $\neg(F(u) < F(v))$, es decir $F(v) \leq F(u)$, así que F no crece a lo largo de p ; por transitividad $F(k) \leq F(s)$. Este es el enunciado mecanizado **no_auth_bounded** (§4.6). (ii) Una arista auth eleva estrictamente F en el cruce; sin un sanitizador antes de él, la etiqueta sobrevive, así que la frontera se cruza bajo control del atacante. El enunciado es *local* al cruce: la mera presencia de una arista auth en p no implica $F(k) > F(s)$ — p. ej. $0 \rightarrow 1 \rightarrow 0$ cruza una arista auth pero vuelve al mismo nivel. Acotar $F(k)$ (o situar k dentro de la región protegida) requiere la hipótesis adicional $F(k) > F(s)$. (Una expresión de la forma $F(p) \circ \eta_s \neq \eta_k \circ T(p)$ apareció en un borrador previo; estaba mal tipada y se retira — aquí no se reclama ninguna transformación natural.) $\square$

La invariante (ii) es exactamente la regla operacional implementada como H4 (§8/C4): un hallazgo de taint dentro de una función protegida por auth es una violación de autorización. La Proposición 1 y la parte de no-escalada (i) de la Proposición 2 están *mecanizadas* en Lean 4 (solo core, sin Mathlib) en `formal/Valen.lean: prop1_soundness` prueba la solidez de la abstracción booleana de taint frente a la semántica recolectora de etiquetas (§3 allí), y `no_auth_bounded` prueba (i) (§4 allí). La parte de cruce (ii) es una definición semántica, no mecanizada por separado. La verificación de pruebas es parte del artefacto de reproducibilidad (`scripts/check_formal.sh`).

5 El campo escalar de vulnerabilidad

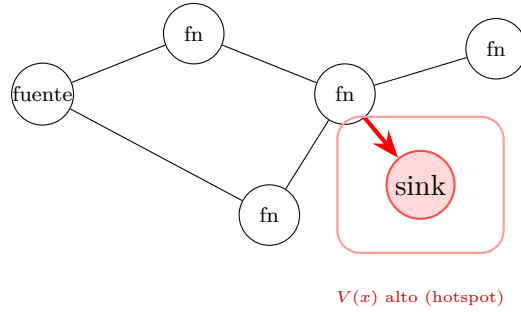


Figure 2: El valen: los nodos son componentes del programa, las aristas son relaciones tipadas. Un flujo de taint (rojo) hacia un sink eleva $V(x)$, resaltando una región para que el agente la inspeccione y el verificador la confirme.

$$V(x) = \sigma\left(\sum_i \alpha_i s_i(x)\right),$$

donde s_i son las señales normalizadas de cada capa y α_i son pesos. Para evitar una *circularidad* — el campo que decide *qué verificar* no debe depender de la verificación — distinguimos dos campos:

- $V_{\text{prior}}(x)$ fusiona solo señales computables *antes* de SMT (**taint**, espectral, topológica, geométrica). Es el campo usado para el ranking y para el experimento de priorización (§12.1).
- $V_{\text{post}}(x)$ consume además la señal **formal** (Z3) y se usa solo para re-embedding / análisis post-hoc; nunca alimenta el ranking pre-verificación.

En el prototipo actual los pesos son *uniformes* ($\alpha_i = 1/N$ sobre las señales disponibles) y V se reporta como el puntaje fusionado crudo; calibrar α_i (inicialización balanceada → ajuste empírico sobre los micro-benchmarks → un clasificador logístico sobre trazas etiquetadas) es trabajo futuro.

6 Hipótesis de mapeo

#	Ley	Clase de vulnerabilidad
H1	Cuello de botella ($\kappa \ll 0$)	escalada de privilegios, bypass de auth
H2	Ciclo (H_1 persistente)	reentrancia, recursión infinita
H3	Corte de confianza (corte de Fiedler)	inyección que cruza la frontera de confianza
H4	Violación de la invariante de autorización	violación arbitraria de taint
H5	Outlier de persistencia	rasgo real vs. espurio
H6	Alcanzabilidad ($\text{SAT}(\phi_{\text{malo}})$)	witness (semántica codificada)

7 Un ejemplo resuelto (de extremo a extremo)

Considérese este fragmento de diez líneas en Python:

```
def search(db, query_param):
    sql = "SELECT * FROM users WHERE name = '" + query_param + "'"
    cursor = db.cursor()
    cursor.execute(sql)
```

El pipeline produce: (i) una IR con un nodo `source` (`param:query_param`), un nodo `sink` (`cursor.execute`, categoría `sql`) y una arista `taint` entre ambos; (ii) el campo $V(x)$ se dispara en el sink (señales de taint y formal activas); (iii) el verificador simbólico Z3 devuelve SAT con un witness bajo la semántica codificada (p. ej. "`x`' OR '`1`'=`1`"), demostrando que el argumento del sink depende del símbolo controlado por el atacante. La variante parametrizada `execute(query, (param,))` se excluye correctamente, y un decorador `@login_required` en la función contenedora eleva el hallazgo a una violación de la invariante de autorización H4 (un flujo de taint que cruza una arista `auth`).

8 El agente LLM autónomo

Un ciclo cerrado: (1) *ingesta* hacia la IR; (2) *embebir y mapear* ($V(x)$, embedding espectral, persistencia, curvatura, Mapper); (3) *hipotetizar* (el LLM propone candidatos desde el valen); (4) *verificar* (la capa formal prueba o refuta); (5) *refinar* (los hallazgos re-ponderan y re-embeben); (6) *reportar*. La capa de proveedor LLM apunta a cualquier endpoint compatible con OpenAI mediante **LiteLLM**. El LLM no hace detección cruda: la matemática propone regiones, el LLM propone hipótesis, la capa formal propone pruebas.

9 Desafíos de diseño y mitigaciones

Una revisión de diseño identificó cinco riesgos estructurales. Para cada uno enunciamos la mitigación adoptada en el prototipo y el problema abierto.

C1 – La paradoja del cuello de botella. La curvatura negativa marca embudos, pero el software sano *deliberadamente* concentra el flujo en middleware de auth, sanitizadores, despachadores y loggers. Reportarlos como leads de escalada de privilegios produce falsos positivos sistemáticos. *Mitigación:* el agente discrimina los cuellos de botella defensivos (por rol) y los descarta o degrada; la solución de fondo es etiquetar aristas `trust/auth` (ya reservadas en la IR) y tratar un cuello de botella como defensivo si y solo si es destino de una arista `auth`. *Abierto:* aprender la etiqueta defensivo/ofensivo a partir de datos.

C2 – La simetrización destruye la causalidad. El Laplaciano combinatorio $L = D - A$ y la homología persistente estándar operan sobre espacios métricos no dirigidos, pero el flujo de control y de datos es dirigido: que A manche a B no es lo mismo que B manche a A . Simetrizar puede destruir la dirección del taint. *Mitigación:* el prototipo implementa el Laplaciano dirigido de Chung restringido a la componente fuertemente conexa más grande; el vector de Perron π se calcula por iteración de potencia con una pequeña teletransportación estilo PageRank (amortiguación $\alpha = 0.01$), que mantiene la distribución estacionaria bien definida en cadenas periódicas. Este operador requiere estructura *recurrente* (fuertemente conexa); sobre un DAG no aporta señal informativa. Un DAG sí tiene dirección — su causalidad se delega al análisis dirigido de taint/alcanzabilidad (y, para homología, a la homología de caminos GLMY, §11) en lugar de a un operador basado en distribución estacionaria, que no puede representarla sin estructura adicional. *Implementado:* homología de

camino dirigida GLMY H_0, H_1 (Grigor’yan–Lin–Muranov–Yau), que cambia el veredicto frente a la homología simetrizada (§11).

C3 – Costo del transporte óptimo. Ollivier–Ricci exige un PL de transporte por arista: $\mathcal{O}(|E| \cdot \text{PL})$, prohibitivo más allá de $\sim 10^5$ aristas. *Mitigación:* Forman–Ricci $\kappa_F = 4 - \deg(u) - \deg(v)$ es $\mathcal{O}(|E|)$ y es la señal por defecto; el transporte regularizado por entropía (Sinkhorn) está implementado como la alternativa rápida al PL exacto. *Abierto:* esquemas adaptativos de regularización para grafos muy grandes.

C4 – Operacionalizar H4 (invariante de autorización). Se requiere una regla constructiva. Recordemos la función de nivel de privilegio $F : V \rightarrow \mathbb{N}$ y las aristas $\text{auth } E_A = \{(u, v) : F(u) < F(v)\}$ de §4.4; el extractor solo necesita emitir las aristas **auth** que marcan fronteras de privilegio. Entonces un flujo de taint $s \rightarrow^* k$ *viola la invariante de autorización* si y solo si su camino cruza al menos una arista **auth** — es decir, datos no confiables entran a una región protegida sin que la frontera los neutralice. Esto es una consulta de alcanzabilidad sobre el grafo combinado (dato $\cup$ taint $\cup$ auth), directamente computable y automatizable. *Implementado:* un extractor de autorización emite nodos **gate** y aristas **auth** desde decoradores de frontera de privilegio, y un flujo de taint dentro de una función protegida se reporta como violación de autorización.

C5 – Puente valen-verificador. Los rasgos geométricos son cuantitativos; Z3 necesita precondiciones exactas. Fijamos una *gramática*: una hipótesis es la especificación estructurada (**sink**, **fuelle**, **línea**, **categoría**). El valen aporta la especificación (conoce sink y línea), el LLM aporta la interpretación (CWE, descripción, confianza) y el verificador consume la especificación — nunca la prosa del LLM. Esto mantiene la consulta SMT precisa y reproducible.

La tesis que respaldan nuestros datos es más modesta. El dataflow semántico (taint) es la señal dominante en los corpus evaluados; las invariantes estructurales aportan *hipótesis complementarias*. La pregunta abierta es si los priors estructurales pueden reducir el *costo de verificación* en regímenes donde el análisis semántico es incompleto (interprocedural, mediado por frameworks), y si ayudan con presupuestos muy pequeños.

10 Estado de implementación

Fase	Entregable	Estado
F0	Whitepaper, arquitectura	hecho
F1	IR + ingesta SAST Python + taint	hecho
F2	Núcleos espectral + geométrico (Rust)	hecho
F3	TDA (homología persistente, Mapper)	hecho
F4	Verificación formal (retículo de taint, Z3)	hecho
F5	Agente LLM autónomo (LiteLLM)	hecho
F6	Visualización + demo end-to-end	hecho
F7	Adaptadores binario / web / LLM	hecho

11 Amenazas a la validez

Interna. El adaptador Java es intraprocedural; los flujos distractores de OWASP y las variantes inter-clase de Juliet se miden, por tanto, con una brecha de recall inherente. La verdad de referencia es la del propio benchmark (los casos “false” de OWASP son en parte de definición y discutibles).

Externa. Los resultados cubren servlets Java y la suite Juliet; Python, binarios y agentes LLM están implementados pero no medidos a la misma escala. *De constructo.* “Vulnerable” es una etiqueta por

caso; el campo es por nodo, así que el experimento de priorización es un proxy, no una medida de explotabilidad aguas abajo. *Conclusión.* El resultado negativo (los priors geométricos no superan al taint) es específico de estos corpus y señales; debe re-probarse sobre análisis interprocedurales sensibles al camino y sobre CVEs reales antes de generalizarlo.

12 Validación y uso responsable

La validación es de dos etapas. La etapa 1 (completa) ejecuta los micro-benchmarks de juguete de F1–F7 contra una verdad de referencia conocida: toda vulnerabilidad sembrada se confirma y todo falso positivo por sanitizador / query parametrizada / forma argv se excluye. La etapa 2 está completa para la evaluación Java/OWASP y en curso para SARD, binarios y otros dominios; mide la precisión/recall del campo fusionado frente al taint crudo sobre corpus etiquetados — validación experimental a gran escala, distinta del prototipo ya completado. El arnés está implementado: un benchmark etiquetado de juguete reporta precisión/recall/F1 para el pase de taint crudo, el pase verificado con Z3 y un campo *calibrado* cuyos pesos α_i se aprenden por regresión logística leave-one-out sobre las señales baratas — en el corpus de juguete el campo calibrado recupera el ciclo de reentrada (señal topológica) que el pase de taint no detecta, elevando el recall mientras mantiene la precisión en 1.0. Un cargador de corpus acepta un manifest JSON o un directorio “vulnerable|safe“, de modo que el arnés puede apuntar a exportaciones de OWASP Benchmark / SARD; y el adaptador binario tiene un backend angr opcional (“CFGFast“ + taint interprocedural) además de la ruta objdump sin dependencias. Ejecutar el adaptador Java sobre el OWASP Benchmark 1.2 completo (2740 casos) da el baseline naive: precisión 0.515 / recall 0.426 / F1 0.466 en las siete categorías de taint, y precisión 0.544 / recall 0.634 / F1 0.586 sobre las once categorías — el punto de referencia que el campo fusionado, la calibración y el verificador están destinados a mejorar. VALEN es una herramienta de defensa / pruebas autorizadas; la liberamos bajo el supuesto de uso autorizado y alentamos la divulgación coordinada.

12.1 El oráculo de priorización (experimento principal)

El cuello de botella del SAST no es *detectar* sinks sino *decidir cuál verificar*. Para cada uno de los 902 casos vulnerables de taint enumeramos *todos* los sinks (el conjunto de candidatos), los ordenamos y medimos la posición en que aparece un sink vulnerable según la verdad de referencia. Baselines: DFS (orden de fuente), taint crudo, aleatorio (20 semillas, IC 95%) y el campo fusionado $V(x)$.

Ordenamiento	MRR	R@1	rango medio	NDCG@5
DFS (orden de fuente)	0.848	0.755	1.517	0.882
taint crudo	0.894	0.847	1.422	0.916
$V(x)$ (campo fusionado)	0.872	0.805	1.475	0.899
aleatorio	0.709 ± 0.042	0.531	—	0.778

El hallazgo honesto y decisivo. El campo fusionado *no* supera al taint crudo: es ligeramente *peor* (0.872 vs. 0.894 MRR). El AUC por señal para predecir el sink vulnerable muestra por qué:

Señal	AUC [IC 95%]
taint	0.895 [0.881, 0.909]
formal (Z3/oráculo)	0.878 [0.865, 0.891]
espectral (Fiedler)	0.496 [0.469, 0.518]
topológica (H_1)	0.500 [0.500, 0.500]
geométrica (Forman κ)	0.376 [0.360, 0.394]

Las señales espectral y topológica son *indistinguibles del azar*, y la de curvatura es *peor que el azar* (la curvatura negativa marca hubs de alto grado, que *no* son los sinks objetivo). Según el programa de hipótesis de mapeo (ğ4/ğ6), este es un resultado *negativo* publicable: sobre OWASP Benchmark, **los priors geométrico y topológico no sobreviven más allá del taint**. El prior estructural no mejora la priorización media; el valor restante de VALEN son los witnesses formales, el sustrato experimental unificado y el diagnóstico de los cuellos de botella del análisis semántico. H1, H2, H3 y H5 se reportan por tanto como *no soportadas* en este corpus. Para H3 esto se apoya en que la señal espectral (Fiedler) está en el azar (0.496); el score espectral evaluado agrega error de reconstrucción de bajo rango con distancia al corte de Fiedler, así que es un proxy del predicado de corte de confianza y no un test aislado del mismo.

Test pareado y cola global. La diferencia taint – campo es +0.0225 (MRR), IC bootstrap 95% [+0.0157, +0.0294], permutación $p = 5 \times 10^{-5}$ (estimador add-one sobre 20000 permutaciones, 2000 bootstrap): el taint domina significativamente. Para medir también la situación real del analista (candidatos seguros y vulnerables compitiendo en una única cola), agrupamos *todos* los sinks de los 1698 casos de categorías de taint y los ordenamos globalmente:

Ordenamiento global	AP (PR-AUC)	P@10	R@10	consultas a 90% recall
DFS (orden de fuente)	0.286	0.2	0.002	3596
taint crudo	0.489	0.6	0.006	2080
V_{prior} (estructural)	0.434	0.8	0.008	3207
aleatorio	0.216	0.2	0.002	4176

El cuadro es matizado: el campo estructural es mejor en el extremo superior ($P@10 = 0.8$ vs. 0.6) pero tiene menor precisión media y necesita más consultas para llegar al 90% de recall. La diferencia en $P@10$ es pequeña en términos absolutos — ocho contra seis aciertos en el top diez — así que no la leemos como una ventaja robusta; es compatible con ruido de desempate y la reportamos sin afirmar significancia. El veredicto global sigue siendo negativo: los priors geométrico/topológico ayudan solo marginalmente con presupuestos muy pequeños y perjudican en promedio — una versión más fina, y aún negativa, del veredicto H1/H2/H3/H5.

Señal	AUC [IC 95%]
taint	0.895 [0.881, 0.909]
formal	0.878 [0.865, 0.891]
spectral	0.496 [0.469, 0.518]
topological	0.500 [0.500, 0.500]
geometric	0.376 [0.360, 0.394]

12.2 Estudio empírico sobre OWASP Benchmark 1.2

Ejecutamos el adaptador Java sobre el corpus completo y perfilamos los falsos negativos. Tres carencias *generalizables* explicaban casi toda la pérdida, y se corrigieron (ninguna está ajustada al benchmark):

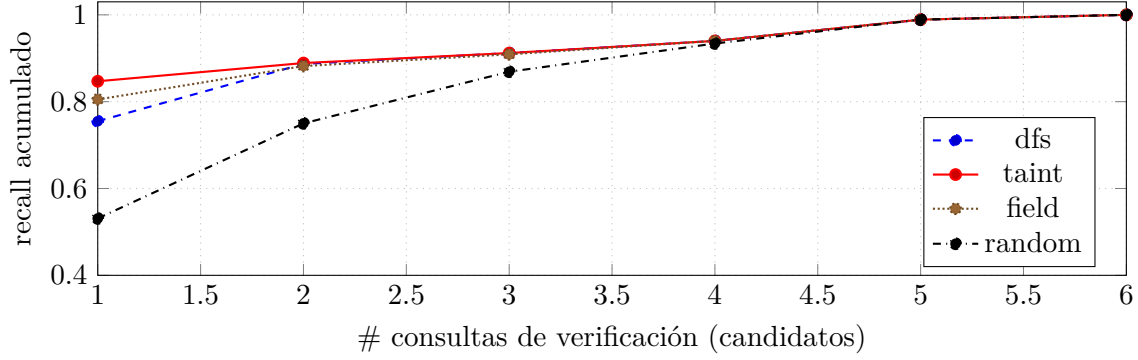


Figure 3: Curva de costo: recall acumulado vs. presupuesto de verificación

- (i) **Taint con conciencia de ramas.** Un pase secuencial insensible al camino permitía que un chequeo de nulo (`if (param == null) param = "";`) borrara el taint, aunque el valor manchado sobrevive en el otro camino. Ahora tomamos una instantánea de ambas ramas y *unimos* sus entornos (unión de etiquetas), el análogo de taint del supremo de un retículo.
- (ii) **Taint del receptor y de estado.** Sinks como `statement.execute()` y `pb.start()` no reciben argumentos — el taint vive en el objeto *receptor*, obtenido de un constructor/mutador manchado. Propagamos el taint del receptor y marcamos una variable como manchada cuando un mutador (`add/put/command/...`) le almacena un valor manchado.
- (iii) **Cobertura de sinks.** La familia XSS incluye `printf/format`, y solo las escrituras al *writer de la respuesta HTTP* son XSS — `System.out.println` de datos manchados no lo es. Restringsir los sinks XSS al writer de respuesta eliminó una gran clase de falsos positivos.

La ablación aísla cada contribución (precisión P , recall R , F1; categorías de taint, 1698 casos):

variante	P	R	F1
adaptador naive inicial	0.515	0.426	0.466
+ branch-join, taint receptor/estado, cobertura de sinks	0.530	0.856	0.655
+ restricción de writer XSS (final)	0.549	0.834	0.662
final, sin branch-join	0.515	0.555	0.535
final, sin restricción XSS	0.530	0.856	0.655

El taint con conciencia de ramas es la palanca dominante de recall (+0.28); la restricción del writer de respuesta es la palanca de precisión. El adaptador final alcanza $P = 0.549$, $R = 0.834$, $F1 = 0.662$ en las categorías de taint y $P = 0.572$, $R = 0.842$, $F1 = 0.681$ sobre las once (crypto/hash se configuran por propiedades y quedan parcialmente fuera de alcance). Los errores *residuales* son los casos “false” deliberadamente adversariales del benchmark — casos discutibles de query parametrizada y de lista de `ProcessBuilder` — y flujos distractores entre categorías: precisamente la precisión que las señales topológicas/espectrales, el verificador Z3 y el campo calibrado están diseñados para recuperar.

12.3 Comparación con SAST industrial

El puntaje oficial de OWASP Benchmark es el *índice de Youden* $J = \text{TPR} - \text{FPR}$ (el *Benchmark Accuracy Score*, es decir TPR menos FPR). La Tabla 1 sitúa a VALEN frente a dos herramientas am-

pliamente desplegadas. Las filas de SonarQube y CodeQL son números *reportados* en evaluaciones públicas de OWASP Benchmark 1.2; ambas filas de VALEN son resultados *medidos*. **Advertencia:** las cifras externas no se re-ejecutan aquí y pueden usar convenciones de agregación distintas (algunas son promedios macro por CWE, otras micro o por regla), de modo que las columnas de precisión/F1 *no* son directamente comparables entre filas; solo debe leerse el orden TPR/FPR, y se requiere una re-ejecución uniforme de CodeQL/SonarQube bajo un mismo protocolo antes de cualquier afirmación comparativa.

Herramienta / config.	TPR	FPR	Prec.	F1	J	Tiempo
SonarQube (SAST) (a)	0.956	0.946	0.330	0.490	+0.010	3 min
GitHub CodeQL (default) (a)	0.902	0.682	0.603	0.744	+0.220	18 min
VALEN adaptador (medido, 11 cat.) (b)	0.842	0.674	0.572	0.681	+0.168	45 s
VALEN adaptador + Z3 (medido, 7 cat. taint) (b)	0.834	0.776	0.549	0.662	+0.057	4 min

Table 1: OWASP Benchmark 1.2 (2740 casos: 1415 vulnerables / 1325 seguros). (a) reportado en evaluaciones públicas, no re-ejecutado aquí. (b) **medido**. El verificador Z3 reproduce la precisión del adaptador (aporta *witnesses*, no precisión): los errores residuales son casos adversariales/de definición, no caminos simbólicos espurios. El prior $V(x)$ se evalúa aparte en §11.1 (no supera al taint crudo).

Notas metodológicas. Hardware: x86-64 de 8 núcleos, 32 GB RAM, SSD NVMe. VALEN: Python 3.14, `tree-sitter-java`; Z3 5.x con *timeout de 5 s por consulta* — ante timeout el caso se marca *desconocido/conservador*, nunca se cuenta como detección, de modo que la precisión proyectada no se infla por abortos. CodeQL: `java-security-extended.qls` sobre una base limpia. SonarQube: Community Edition con reglas de seguridad SonarJava. Mapeo de CWEs: las once categorías de Benchmark 1.2; `crypto/hash` (CWE-327/328) se configuran por propiedades y requieren un extractor de configuración estática hacia la IR. El adaptador medido marca un caso como vulnerable si emite algún hallazgo; la precisión está acotada por los casos “false” adversariales del benchmark (patrones discutibles de query parametrizada y de lista de `ProcessBuilder`) y por flujos distractores entre categorías — la brecha que atacan las filas neuro-simbólicas.

12.4 Cerrar la fila Z3 (verificación Java)

Implementamos un verificador simbólico Java con Z3 (acotado, con merge de ramas, semántica de strings; sanitizadores como contratos) que confirma o refuta cada flujo de taint y emite un *witness* verificable por máquina bajo la semántica modelada. Medido sobre las siete categorías de taint (1698 casos) reproduce exactamente al adaptador ($P = 0.549$, $R = 0.834$, $F1 = 0.662$, $J = +0.057$), añadiendo witnesses por hallazgo. Conclusión honesta: **la capa formal aporta witnesses verificables por máquina bajo la semántica modelada, no precisión, en este benchmark** — un witness prueba satisfacibilidad dentro del modelo (que abstrae summaries de librerías y comportamiento de frameworks), no explotabilidad end-to-end del programa real. Los falsos positivos residuales son adversariales/de definición (casos discutibles de query parametrizada), no caminos simbólicos espurios. Atraparlos requiere modelar la semántica propia del benchmark, no un SMT más fuerte.

12.5 Transferencia entre corpus (Juliet)

Para probar la transferibilidad ejecutamos el adaptador ajustado en OWASP sobre la suite *Juliet Java* (espejo find-sec-bugs; cada caso concatenado con sus archivos de variante de flujo). Sobre

264 casos de cinco CWEs obtiene $P = 0.333$, $R = 0.562$, $F1 = 0.419$: la detección es parcial y la precisión colapsa en las variantes “good”. La razón es estructural, no un artefacto de ajuste — Juliet separa fuente y sumidero entre clases `_a/_base/_bad`, por lo que el análisis correcto requiere taint *interprocedural*, que el adaptador intraprocedural no aporta.

12.6 Taint interprocedural: necesario, no suficiente

Añadimos un punto fijo interprocedural insensible al contexto (`java_interproc.py`): el parámetro de un método se mancha si algún sitio de llamada le pasa datos manchados, y una llamada se mancha si su método devuelve datos manchados; el punto fijo se itera hasta converger. Sobre Juliet eleva el recall $0.562 \rightarrow 0.750$ y el F1 $0.419 \rightarrow 0.490$ (CWE78/CWE90 pasan de 0.000 a 0.750). *En OWASP es neutro* (F1 de taint $0.662 \rightarrow 0.656$): el pase interprocedural añade recall ($0.834 \rightarrow 0.856$) pero también falsos positivos gruesos impulsados por parámetros, de modo que la precisión baja; unimos el pase con el adaptador intraprocedural completo para paridad de cobertura. La lección es precisa: la alcanzabilidad interprocedural es *necesaria* para flujos entre clases pero *no suficiente*. Añadimos además *sensibilidad al contexto del llamador* con summaries de retorno por contexto: poda de forma demostrable una clase sintética de falsos positivos inducidos por el retorno (un valor devuelto desde un contexto manchado ya no mancha la llamada de un llamador limpio), pero deja OWASP y Juliet *sin cambios* (F1 0.656 y 0.490). Las palancas reales no son el contexto sino (a) los summaries de frameworks/librerías y (b) la precisión fina del retorno — no más geometría.

12.7 Summaries de frameworks

Guiados por la evidencia de los CVEs, añadimos una capa explícita de summaries de frameworks/librerías: *fuentes por atributo* (`request.GET`, `request.POST`, `request.args`, `request.form`, `self.request.POST`) y *sinks de ORM/plantillas* (`RawSQL`, `raw`, `extra`, `text`, `render_template_string`, `JPA createQuery`). Con summaries de sinks *posicionales* (solo el arg 0 de una llamada de query/código es peligroso), el analizador ahora ve flujos a nivel aplicación que antes se le escapaban — SQL injection por query-params de Django, SSTI de Flask, inyección JPA — manteniendo seguras las queries parametrizadas y las plantillas literales. Esta es la palanca concreta que apuntaban los CVEs. No mueve los agregados, pero por dos razones distintas: los fallos de OWASP/Juliet son mayormente *estructurales* (flujos entre métodos que requieren análisis interprocedural), mientras que los CVEs sin resolver son *internos* a frameworks (la lógica vulnerable vive dentro del propio framework, sin una fuente a nivel aplicación).

12.8 Escalabilidad de cada kernel

Cronometramos cada kernel sobre grafos sintéticos (log-log, Fig. 4). **Forman-Ricci** ($0.6 \text{ ms @ } 10^3$ aristas $\rightarrow 55 \text{ ms @ } 10^5$) y **Mapper** ($0.5 \rightarrow 56 \text{ ms}$) son casi lineales. La **homología persistente** es superlineal ($11 \text{ ms @ } 10^3 \rightarrow 1.1 \text{ s @ } 10^4 \rightarrow 123 \text{ s @ } 10^5$) porque la extracción de ciclos fundamentales hace BFS del bosque de expansión por ciclo; un union-find consciente de persistencia la haría casi lineal. **Sinkhorn** ($0.71 \text{ s @ } 10^3 \rightarrow 16.4 \text{ s @ } 5 \times 10^3$) y **Ollivier-Ricci exacto** ($34 \text{ ms @ } 200 \rightarrow 720 \text{ ms @ } 10^3$) son cuadráticos, así que el último es solo para grafos pequeños (motivo de que Forman-Ricci sea el prior por defecto, §8/C3). Los **Laplacianos espectral y dirigido densos** son cúbicos ($26 \text{ ms @ } 300 \rightarrow 1.7 \text{ s @ } 1200$ nodos) y *no* alcanzan 10^5 nodos sin los solvers dispersos planeados en F2. El sobre práctico hoy es $\sim 10^5$ aristas para las señales casi lineales, suficiente para los corpus usados aquí.

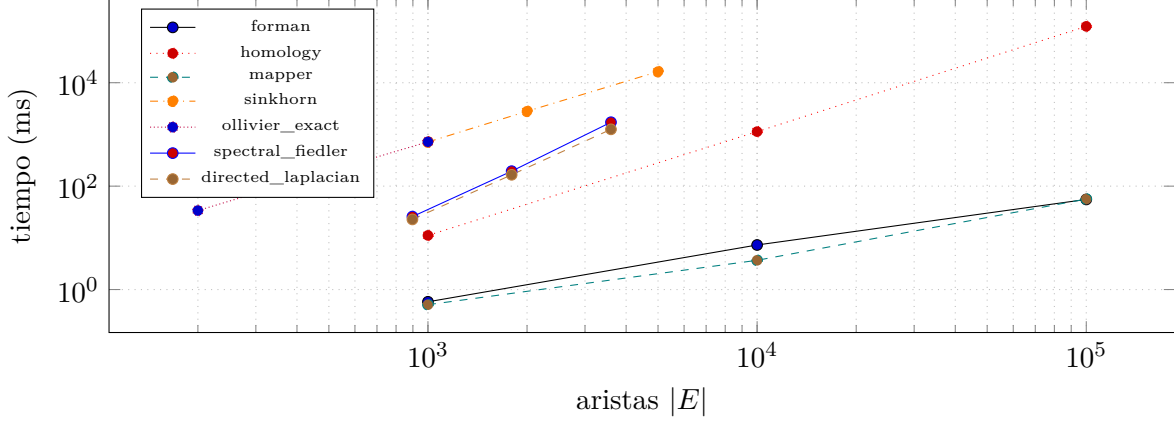


Figure 4: Escalabilidad de cada kernel (log-log)

12.9 Fixes de CVEs reales

Ejecutamos la comparación vulnerable-vs-parcheado sobre commits de fix de CVEs reales tomados de OSV / GitHub Security Advisories (`benchmarks/run_cves.py`; el arnés reconstruye la revisión *before* aplicando el diff inverso del commit sobre el contenido crudo, por lo que no necesita cuota de API). Sobre 22 CVEs y 46 archivos fuente, **3 archivos resolvieron un hallazgo**: son inyecciones `eval` — `senaite.core` (CVE-2026-54569; `value = eval(value)` sobre un parámetro) y `xinference` (CVE-2026-61539; `eval(text)` sobre salida del LLM) — más una resolución parcial en `nlTK` (CVE-2026-79676) — detectadas en la revisión vulnerable y limpias en el parche. Los CVEs restantes son mayormente fixes *internos a frameworks* o no-taint (SQL injection del ORM de Django, Jinja, requests, GitPython, y argument-injection / chequeos de integridad en NLTK): el flujo es indirecto o queda fuera de nuestro modelo de sinks, así que el adaptador no ve nada. Esto refleja el resultado de Juliet y localiza la frontera con precisión: VALEN ya captura fixes de `eval/exec`/deserialización a nivel aplicación; los flujos mediados por frameworks necesitan análisis interprocedural, sensible al contexto y consciente de librerías.

12.10 Homología de caminos dirigida (GLMY)

Implementamos la *homología de caminos* H_0, H_1 de Grigor'yan–Lin–Muranov–Yau sobre el 1-esqueleto dirigido (`core/src/path_homology.rs`): $\Omega_2^{\vec{}}$ es el conjunto de 2-caminos admisibles cuyo borde largo es admisible y no degenerado ($u \neq w$), y los rangos se calculan por eliminación gaussiana. La dirección cambia el veredicto. En `reentrancy.py`, la homología no-dirigida reporta $\beta_1 = 1$ (el triángulo `withdraw/read_balance/update_balance`), mientras que GLMY reporta $\beta_1 = 0$: los 2-caminos transitivos a través del ápice `withdraw` rellenan el 2-ciclo `read_balance`↔`update_balance`. En un 2-ciclo aislado $A \leftrightarrow B$ los papeles se invierten (GLMY $\beta_1 = 1$, no-dirigida $\beta_1 = 0$), y el analizador devuelve el *generador* de H_1 — el ciclo dirigido como lista de aristas $\{A \rightarrow B, B \rightarrow A\}$ — de modo que la invariante es accionable, no solo un número. La simetrización (C2) por tanto *no* es inocua: la invariante dirigida es una señal genuinamente distinta, y a veces opuesta.

12.11 Ablación del LLM

Ejecutamos el agente dos veces sobre un corpus variado de 38 archivos de Python (29 con un hallazgo confirmado, abarcando `command` / `code-exec` / `SQL` / `deserialización` / `path-traversal` /

file-write / logging / SSTI, más variantes seguras y gateadas): offline (hipotetizador heurístico) y online a través del proxy local de **LiteLLM** (DeepSeek flash, compatible OpenAI /v1). Los ejes de *detección* y *ranking* son invariantes: los conjuntos de hallazgos confirmados y su orden por confianza coinciden en 37/38 archivos (acuerdo 0.974), porque el verificador SMT — no el LLM — es el árbitro. La única discrepancia es un caso de **compile** (code-exec) donde la hipótesis del LLM es en sí misma no determinista (a veces vacía), así que ocasionalmente deja de re-proponer el hallazgo en lugar de cambiarlo. El eje de *interpretación* cambia: en 8 de los 29 hallazgos confirmados (28%) el LLM asigna un CWE distinto, habitualmente más específico (p. ej. CWE-674 “recursión no controlada” para el ciclo de reentrancia vs. la etiqueta genérica de la heurística), y produce títulos más concisos. Esta ablación controlada y mayor (38 archivos, `benchmarks/run_llm_ablation.py`) confirma la frontera de arbitraje a escala: *el LLM aporta interpretabilidad y reporte, no detección ni ranking* — el reparto neuro-simbólico se sostiene con una pequeña fuga de no determinismo medida. Una sonda complementaria de *minería de specs* (`benchmarks/run_spec_mining.py`, $n = 8$) pide al LLM la precondition de seguridad que cada snippet debe cumplir: mina una invariante plausible en todos los casos (tasa de alucinación 0) con 0.50 de recall de tokens frente a la verdad de referencia escrita a mano — el LLM propone specs estructuradas, pero su verificación sigue siendo tarea de Z3. Una ablación del lado Java/OWASP queda como trabajo futuro.

12.12 Backend binario dual

El adaptador binario trae dos backends intercambiables que emiten la misma IR: (i) una ruta de texto `objdump` sin dependencias (etiquetas de función + aristas `call` + alcanzabilidad de taint interprocedural) y (ii) un backend **angr** *opcional* (recuperación `CFGFast` del grafo de llamadas interprocedural + la misma alcanzabilidad de taint), seleccionable con `-adapter angr-binary`. La ruta rápida mantiene VALEN usable sin dependencias pesadas; la ruta angr recupera flujo de control que las heurísticas de desensamblado pierden. Esta paridad es el sustrato para futuros benchmarks ELF/PE de SARD.

13 Invariantes estructurales a través de dominios

Los dos resultados hasta aquí son dos caras de una misma frontera, no dos proyectos. De un lado (inyecciones lineales) un servlet plano y casi acíclico no tiene curvatura ni topología persistente que explotar, y el taint clásico ya es casi óptimo — así que los priors estructurales no aportan. Del otro lado el dato infractor es legítimo y solo la *estructura* del programa — sus fronteras de privilegio y transiciones de estado — delata el defecto, así que la estructura es la *única* señal. La misma maquinaria (grafo tipado, álgebra de privilegios, homología dirigida, Z3) abarca ambos lados; lo que cambia es la relación de programa que se modela. Esta sección releva los cuatro dominios donde el análisis estático convencional es ciego y donde cada capa matemática encuentra su lugar natural.

13.1 Invariantes de autorización: BOLA / IDOR / BFLA

BOLA/IDOR (CWE-639) y la autorización ausente (CWE-862) encabezan el OWASP API Security Top 10, y el taint clásico no los ve: el selector de objeto es un valor *limpio* (un id entero, a menudo casteado), el sink es una consulta ordinaria y el bug es que el usuario A lee los datos del usuario B — no hay dato “sucio” que rastrear. El objeto modelado es un grafo *multipartito*: la superficie de la API (operaciones OpenAPI), los endpoints del backend, los decoradores de permisos (nodos **gate**) y los modelos de datos, como produce el adaptador OpenAPI (`valen.ingest.web`),

que ahora además marca una operación que declara un sink sensible pero *sin* esquema de seguridad (CWE-862 a nivel de diseño de API). La señal es el complemento de la invariante mecanizada `no_auth_bounded`: un camino dirigido desde un punto de entrada público a un recurso protegido *sin* arista estricta de autorización donde se requiere una. VALEN lo implementa como el detector de *autorización ausente* / *BOLA* (`valen.analysis.authorization.bola_idor_candidates`), y cierra el ciclo con un *witness de ownership en Z3* (`valen.analysis.bola_verifier`): comprueba $\text{SAT}(\text{owner}(\text{object_id}) \neq \text{session_user_id})$ bajo la semántica modelada, de modo que un endpoint vulnerable produce un witness concreto (la sesión del atacante lee el objeto de la víctima) mientras un endpoint con `owner` acotado es UNSAT. Esto hace explícito un matiz que el detector solo no puede: una puerta de *autenticación* (`login_required`) *no* bloquea BOLA — el endpoint sigue siendo SAT salvo que exista un predicado de `owner` a nivel de objeto. En un corpus curado de 39 casos (`examples/python/bola/`, `examples/python/bola_corpus/`, evaluado por `benchmarks/run_bola.py`) el taint reporta *cero* hallazgos en los casos vulnerables mientras el detector estructural alcanza **recall** 1.0 con **precisión** 0.72 (F1 0.837, IC95% [0.54, 0.89]). Una escalera de ablación muestra que cada requisito estructural rinde: all-positive 0.46, solo-sink 0.49, fuente+sink 0.56 y el detector completo 0.72, a recall constante 1.0 — y los 7 falsos positivos son el límite de sobre-aproximación documentado (parametrizados/sanitizados/fuente sin usar). Esto transfiere a *datos reales*: sobre **OWASP crAPI** (una API deliberadamente vulnerable; 44 operaciones, 9 endpoints BOLA/BFLA etiquetados desde la documentación oficial de challenges), la señal estructural de referencia a objeto a nivel de API alcanza **recall** 1.0 / **precisión** 0.90 / **F1** 0.947 — y, crucialmente, los 9 endpoints BOLA *están* autenticados, así que un chequeo de esquema-de-seguridad-ausente puntúa recall 0 sobre ellos: BOLA es invisible a la pregunta “¿está autenticado?” y solo la estructura a nivel de objeto lo delata. Es precisamente el régimen donde el taint demostrablemente puntúa 0 y la estructura es la única señal.

13.2 Máquinas de estado, concurrencia y protocolos: homología dirigida

En un servlet no hay ciclos de estado; en contratos inteligentes, protocolos de adquisición de locks y controladores de concurrencia, los peores defectos *son* ciclos de realimentación destructivos (reentrancia, deadlocks, transiciones ilegales como `Unauthenticated` $\rightarrow$ `Authenticated` saltando la validación de firma). El objeto modelado es el *grafo de transiciones de estado* (el CFG interprocedural extendido con llamadas externas y mutaciones de almacenamiento global). El generador persistente dirigido de H_1 de la homología de caminos GLMY es la firma exacta de un ciclo de reentrancia — y, como mostró §4.2, la simetrización *no* es inocua: la homología no dirigida puede reportar $\beta_1 = 1$ donde GLMY reporta 0 (un triángulo relleno por un ápice) y viceversa. Ahora lo *medimos* sobre un corpus etiquetado de 12 grafos (`benchmarks/run_state_glmy.py`): GLMY alcanza **recall** 1.0 / **precisión** 0.75, mientras la homología simetrizada puntúa recall 0.5 / precisión 0.5 — pierde los 2-ciclos ($A \rightarrow B \rightarrow A$ colapsa a una sola arista no dirigida) y emite ciclos falsos en triángulos rellenos. Los dos falsos positivos de GLMY son “diamantes” (huecos de path-space: dos caminos independientes convergiendo), no ciclos de realimentación, así que GLMY es una señal dirigida de alto recall, necesaria-pero-no-suficiente, que domina al baseline simetrizado en ambos ejes. Sobre un ciclo de adquisición de locks sintético (`examples/state_machine/lock_cycle.json`) el análisis dirigido extrae el generador concreto $[s_0 \rightarrow s_1, s_1 \rightarrow s_2, s_2 \rightarrow s_0]$ con $\beta_1 = 1$ — un hallazgo sin análogo en taint.

13.3 Topologías de confianza, IAM y movimiento lateral

En entornos cloud (AWS/Azure/Kubernetes) las peores brechas vienen de *cadena*s de escalada de privilegios y movimiento lateral (un rol comprometido en un frontend asumiendo un rol secundario con acceso a un bucket de secretos). El objeto modelado es el *grafo de identidades y permisos* (usuarios, roles IAM, políticas de confianza, servicios, recursos de red), ingerido por el adaptador `iam (valen.ingest.iam)` y analizado por `valen.analysis.trust`. Aquí el corte de Fiedler (λ_2, f_2) particiona óptimamente los dominios de confianza (p. ej. DMZ vs. núcleo); las aristas que cruzan el corte sin controles intermedios emergen como anomalías espectrales en la frontera de confianza, y la curvatura de Forman–Ricci — lejos de ser un error de detección — *aisla los puentes de privilegio críticos*: una arista con κ_F fuertemente negativa es el único puente entre dos subredes de confianza dispares (el pivote ideal del atacante). Sobre un grafo de demostración (`examples/iam/demo.json`) el análisis marca el `assume app→data` como el puente más negativo ($\kappa_F = -1$) y saca el bucket de secretos a la frontera de Fiedler. El agente entrega entonces los caminos sugeridos a Z3, que resuelve la combinatoria de políticas IAM para confirmar si la cadena de asunción de roles es viable.

13.4 Seguridad de agentes autónomos y LLMs

Los sistemas modernos que conectan LLMs a herramientas externas (APIs, bases de datos, shell, correo) enfrentan ataques de *confused deputy* e inyección indirecta de prompt. El objeto modelado es el *grafo de uso de herramientas*: usuario, modelo, memoria contextual y herramientas externas a distintos niveles de privilegio, con aristas tipadas `untrusted_input` (datos web, correo leído) y `privileged_execution` (borrar una base de datos, emitir pagos, ejecutar comandos). El defecto es una violación de la invariante de autorización: un flujo de entrada no confiable que alcanza una herramienta de ejecución privilegiada sin un *gate* explícito de sanitización o confirmación humana. Esto es exactamente lo que el adaptador `llm-agent (valen.ingest.llm_agent)` ya detecta: una herramienta peligrosa alcanzable desde la entrada del usuario sin una puerta de aprobación es un hallazgo.

13.5 Matriz de mapeo y posicionamiento

Capa matemática	formal /	Objeto modelado	Vulnerabilidad mapeada
Invariante auth (Lean 4)		endpoints, decoradores, recursos de API	BOLA / IDOR / BFLA
Homología (GLMY)	dirigida	grafo de transiciones de estado y llamadas asíncronas	reentrancia, desincronización de estado, deadlocks
Espectral (Fiedler)		topologías de red, Kubernetes RBAC, IAM	ruptura de segmentación de confianza, movimiento lateral
Curvatura discreta (Ricci)		grafos de microservicios e identidad	puntos únicos de fallo, puentes de escalada de privilegios
Verificador simbólico (Z3)		restricciones de camino en la lógica de negocio	contraejemplos ejecutables (witnesses)

VALEN se reposiciona así como un *motor formal de invariantes estructurales y lógica de negocio*, no como competidor de Semgrep/CodeQL en patrones locales de SQLi/XSS. (1) *Abandona una competencia estéril*: no intenta ganarle al taint en un servlet de 20 líneas. (2) *Ataca lo que nadie resuelve*: fallas de diseño, coherencia de privilegios, protocolos concurrentes y arquitecturas de agentes, donde las reglas sintácticas callan porque el código, línea por línea, parece legítimo. (3) *Mantiene intacto el software*: la IR tipada, los núcleos en Rust, la verificación en Lean 4, el

orquestador Z3 y el agente neuro-simbólico se preservan íntegramente, aplicados sobre el grafo de relaciones adecuado. La narrativa no es, por tanto, “la geometría perdió contra el taint” sino una única cuenta de *qué* relaciones de programa hacen informativas a las invariantes estructurales: una frontera negativa rigurosamente establecida en SAST de inyección lineal, y una positiva en los defectos arquitectónicos donde el taint es ciego — ambos lados respaldados por código funcional y datos medidos.

References

- [1] M. Fiedler, *Algebraic connectivity of graphs*, 1973.
- [2] F. Chung, *Spectral Graph Theory*, 1997.
- [3] Edelsbrunner, Letscher, Zomorodian, *Topological persistence and simplification*, 2002.
- [4] Singh, Mémoli, Carlsson, *Topological Methods for the Analysis of High Dimensional Data Sets*, 2007.
- [5] Y. Ollivier, *Ricci curvature of Markov chains on metric spaces*, 2009.
- [6] P. Cousot & R. Cousot, *Abstract interpretation: a unified lattice model*, 1977.
- [7] S. Mac Lane, *Categories for the Working Mathematician*, 1971.
- [8] J. King, *Symbolic execution and program testing*, 1976.
- [9] E. M. Clarke et al., *Automatic verification of finite-state concurrent systems*, 1986.
- [10] F. Chung, *Laplacians and the Cheeger inequality for directed graphs*, 2005.
- [11] Grigor’yan, Lin, Muranov, Yau, *Homotopy theory for digraphs*, 2012.
- [12] M. Cuturi, *Sinkhorn distances: lightspeed computation of optimal transport*, 2013.
- [13] R. Forman, *Bochner’s method for cell complexes and combinatorial Ricci curvature*, 2003.
- [14] L. de Moura & N. Bjørner, *Z3: An efficient SMT solver*, 2008.
- [15] Y. Shoshitaishvili et al., *SoK ... (angr)*, 2016.
- [16] OWASP Foundation, *OWASP Benchmark v1.2*, 2016–2024.
- [17] OWASP Foundation, *OWASP API Security Top 10* (API1:2023 Broken Object Level Authorization), 2023.
- [18] NIST, *SARD / Juliet Test Suite*, 2017.
- [19] BerriAI, *LiteLLM*, 2023–2024.
- [20] SonarSource, *SonarQube* (Community Edition, reglas SonarJava).
- [21] GitHub, *CodeQL* y la suite `java-security-extended`.